

MIAE 212 · Rapid Review Sheet · Part 2

Operators, Casting, ASCII & Control Flow

1. Type Casting & Conversions

- **Implicit Promotion:** Smaller types automatically widen: `int` $\rightarrow$ `float` $\rightarrow$ `double`.
- **Explicit Cast:** `(int)x` or `static_cast<int>(x)`.
 - Truncates decimal part: `(int)5.999 == 5`.
 - Narrowing overflow: `int y = 3000; char ch = (char)y;  $\implies$  (int)ch = -72` (retains lowest 8 bits).

2. ASCII Character Encoding & Math

Range	Characters	ASCII Dec	Key Mathematical Tricks
Digits	'0' to '9'	48 to 57	Convert char to integer: <code>int val = ch - '0';</code>
Uppercase	'A' to 'Z'	65 to 90	Offset is 32 from lowercase: <code>'a' - 'A' == 32</code>
Lowercase	'a' to 'z'	97 to 122	Convert to upper: <code>char up = ch - 32;</code>

3. Operators & Short-Circuit Evaluation

- **Compound Assignment:** `x += 5` $\iff x = x + 5$; `x *= 2` $\iff x = x * 2$.
- **Increment / Decrement:** Prefix `++i` (increments then yields value) vs Postfix `i++` (yields value then increments).
- **Logical Operators:** `&&` (AND), `||` (OR), `!` (NOT).
- **Short-Circuit Evaluation:**
 - In `A && B`: if A is false, B is **never executed**.
 - In `A || B`: if A is true, B is **never executed**.
 - *Safety Guard:* `if (x != 0.0 && sin(1.0/x) > 0.9)` prevents divide-by-zero crashes.

4. Control Flow & The Floating-Point Rule

- **The Cardinal Rule: NEVER** use `==` with floating-point math! Due to binary representation noise, `1.0 - cos(2*PI) != 0.0`.
 - **Correct:** `if (abs(z - target) < 1.0e-7) { /* practically equal */ }`
- **Immediate Exit:** `exit(0);` (requires `<cstdlib>`) terminates execution instantly from anywhere.

5. Numerical Engineering Algorithms

```
// 1. Target Value Search Loop
for (t = 0.0; t < 1e7; t += 0.1) {
    double f = sin(t);
    if (f > 0.9 && f < 0.93) break; // Terminate early
}

// 2. Numerical Grid-Search Maximization on [0, 10*PI]
double dx = 0.01, m = sin(0 - 1)*(-3) + cos(0), x_best = 0;
for (double x = 0.0; x <= 10 * 3.14159; x += dx) {
    double f = sin(x*x - 1.0) * (x - 3.0) + cos(x);
    if (f > m) { m = f; x_best = x; } // Track highest peak
}
```